

编码任务清单：水泥烧成系统电除尘器协同优化控制

本文档将 spec.md 的需求规格与 design.md 的增量设计方案分解为可执行、可验收的编码任务。所有数学公式与变量均采用 `$$...$$` 包裹。任务按业务功能垂直切割，模块依赖关系为：`data_loader` $\leftarrow$ `modeling` $\leftarrow$ `optim` $\leftarrow$ `sensitivity` / `tighten` $\leftarrow$ `report`。

1. 项目骨架与环境配置

1.1 搭建项目目录结构

- ☐ 在 `Problem/` 下创建模块目录 `data_loader/`、`modeling/`、`optim/`、`sensitivity/`、`tighten/`、`report/`、`tests/`、`outputs/`、`config/`，并各放 `__init__.py` 空文件
- ☐ 创建主入口文件 `Problem/main.py`，预留 `run_pipeline()` 函数骨架，串联各模块调用
- ☐ 创建 `Problem/README.md` 更新为项目运行说明（数据路径、依赖安装、复现命令）

1.2 创建依赖与配置文件

- ☐ 创建 `Problem/requirements.txt`，固定版本：`pandas`、`numpy`、`scipy`、`scikit-learn`、`mysql-connector-python`、`matplotlib`、`pytest`，保证评审可复现（对应 spec 5.5 第 6 点）
- ☐ 创建 `Problem/config/config.yaml`，集中配置：CSV 路径、MySQL 连接串、工况数 $k = 5$ 、排放阈值 $C_{limit} = 10$ 与 $C'_{limit} = 5$ 、随机种子 `seed=42`、寻优算法 `SLSQP`、多起点数 `multi_start=10`、有限差分步长比例 `1%`、输出目录 `outputs/`
- ☐ 创建 `Problem/config/db_schema.sql`，定义 MySQL 表 `esp_raw` / `esp_power_model` / `esp_deutsch_model` / `esp_regimes` / `esp_opt_sol` / `esp_sensitivity` / `esp_tighten`，字段对应 design.md 2.3.2 领域对象

2. 数据加载与预处理（data_loader 模块）

2.1 实现原始数据加载 `load_raw`

- ☐ 在 `data_loader/loader.py` 实现 `load_raw(path) -> DataFrame`，读取 `Cement_ESP_Data.csv` (10080 行 $\times$ 14 列)，强制 dtype (`timestamp` 为 `datetime64[ns]`，其余 `float64`)
- ☐ 校验列数与列名顺序，不符抛 `ValueError`；校验时间戳单调递增、间隔 1 分钟、共 10080 条
- ☐ 报告缺失值统计（预期 C_{out} 有 50 个缺失，其余无缺失），打印基础描述统计

2.2 实现清洗与插补 `clean_and_impute`

- ☐ 在 `data_loader/loader.py` 实现 `clean_and_impute(df, method="time") -> DataFrame`，对 C_{out} 缺失值做时间线性插补
- ☐ 实现异常值标记：基于 3σ 准则标记各字段异常点，记录到 `df.attrs["outliers"]`，不丢弃整段时序（对应 spec 4.1 规则 5 禁止项）
- ☐ 在 `df.attrs["bounds"]` 写入各电场 U_{min} , U_{max} , T_{min} , T_{max} （取历史 min/max 并预留 5% 裕度），振打安全上限 $T_{crit,i}$ 取历史 95 分位数，电压外推上限 $U_{max} \times 1.1$ （对应 design 1.2.4）

2.3 实现 MySQL 持久化 `to_mysql` / `from_mysql`

- ☐ 在 `data_loader/db.py` 实现 `to_mysql(df, table, cfg)` 与 `from_mysql(table, cfg) -> DataFrame`，采用整表替换（`DROP+CREATE+INSERT`）保证复算幂等（对应 design 2.1.3 事务设计）
- ☐ 连接失败时降级为本地 `parquet` 缓存并打印告警，不阻断主流程

3. 问题1 机理建模（modeling 模块）

3.1 实现电耗模型拟合 `fit_power_model`

- ☐ 在 `modeling/power.py` 实现 `fit_power_model(df) -> dict`，构造设计矩阵 $X = [U_1^2, U_2^2, U_3^2, U_4^2]$ ，最小二乘解 $Xk = P_{total}$ ，返回 `{"k": [4], "r2": float, "cond": float}`
- ☐ 强制 $k_i \geq 0$ （物理要求），条件数过大时打印告警
- ☐ 若 $R^2 < 0.9$ ，自动扩展为 $P_i = k_i U_i^2 + b_i U_i + c_i$ 重拟（对应 design 2.1.3 分支触发条件）

3.2 实现 Deutsch 参数拟合 `fit_deutsch_params`

- ☐ 在 `modeling/deutsch.py` 实现 `fit_deutsch_params(df, bounds) -> dict`，拟合各电场合并系数 $k_i A_i$ 与振打衰减系数 α_i ，参考周期 $T_{ref,i}$ 取各电场历史中位数
- ☐ 拟合目标： $C_{out} = C_{in} \cdot 1000 \cdot \prod_{i=1}^4 (1 - \eta_i)$ ，其中 $\eta_i = (1 - \exp(-k_i A_i \cdot U_i^2 / Q)) \cdot \exp(-\alpha_i (T_i - T_{ref,i}))$
- ☐ 采用 `scipy.optimize.curve_fit` 带物理单调性约束（效率随 U_i 单调增、随 T_i 单调减），各电场独立设系数与边界，不混用统一范围（对应 spec 5.2 风险应对、design 1.2.3 约束 2）
- ☐ 拟合后做残差分析与物理合理性检查，不达标打印告警并提示调整模型结构

3.3 实现出口浓度预测 `predict_cout`

- ☐ 在 `modeling/deutsch.py` 实现 `predict_cout(params, Tin, Cin, Q, U, T) -> float`，按上述连乘公式返回 C_{out} 标量
- ☐ 提供 `predict_eta(params, U, T, Q) -> tuple` 辅助函数返回各电场单级效率，供灵敏度分析复用

3.4 实现振打瞬时峰值估算 `predict_peak`

- ☐ 在 `modeling/deutsch.py` 实现 `predict_peak(params, T, Cin) -> float`，半经验关系 $C_{peak} \propto$ 积灰厚度 $\propto$ 振打周期，建立 $C_{peak} = g(T_{1\sim4}, C_{in})$
- ☐ 在函数 docstring 与返回结果中明确标注“基于机理估算，分钟级数据无法直接观测秒级峰值”（对应 spec 5.1 风险、spec 5.5 第 4 点）

3.5 实现特征重要性交叉验证 `feature_importance`

- ☐ 在 `modeling/feature.py` 实现 `feature_importance(df) -> dict`，用随机森林 + 梯度提升计算入口温度/流量/浓度与 $U_{1\sim4}$ 、 $T_{1\sim4}$ 对 C_{out} 的特征重要性排序
- ☐ 同时输出 Pearson 相关系数与显著性 p 值表，作为机理模型的交叉验证（对应 spec 4.1 规则 1/2/4）
- ☐ 在报告中明确说明：因历史 C_{out} 方差仅 0.17，数据驱动结果仅作参考，机理模型为主（对应 spec 5.5 第 2 点）

4. 问题2 工况划分与寻优 (optim 模块)

4.1 实现 K-Means 工况划分 cluster_regimes

- ☐ 在 `optim/regime.py` 实现 `cluster_regimes(df, k=5, seed=42) -> dict`, 以 C_{in}, T_{in} (辅以 Q) 为特征做 K-Means 聚类
- ☐ 返回 `{"regimes": [{id, boundary, n, mean, var}...], "labels": []}`, 校验每工况样本数 $\geq 3\%$ 总样本, 否则减少 k 重聚类
- ☐ 固定 `seed=42` 保证可复现; 输出各工况边界、样本数、特征均值方差表

4.2 实现单工况寻优 solve_one_regime

- ☐ 在 `optim/solve.py` 实现 `solve_one_regime(regime, model, C_limit, algo="SLSQP", multi_start=10) -> dict`, 决策变量 $U_{1\sim 4}, T_{1\sim 4}$ 共 8 维
- ☐ 目标 $\min P_{total} = \sum k_i U_i^2$, 约束: $C_{out} \leq C_{limit}, U_{min,i} \leq U_i \leq U_{max,i}, T_{min,i} \leq T_i \leq T_{max,i}, T_i \leq T_{crit,i}$
- ☐ 振动态约束双重处理: 硬约束 $T_i \leq T_{crit,i} + \text{目标加积灰惩罚项} \lambda \cdot \max(0, T_i - T_{crit,i})^2$ (对应 spec 4.2 规则 4、design 2.1.3)
- ☐ 多起点启动 (10 个) 避免局部最优, 固定随机种子; 返回 `{"U": [4], "T": [4], "P": float, "Cout": float, "success": bool, "n_iter": int, "margin": float}`, $\text{margin} = C_{limit} - C_{out}$ 为约束裕度

4.3 实现批量寻优与交叉验证 solve_all_regimes

- ☐ 在 `optim/solve.py` 实现 `solve_all_regimes(regimes, model, C_limit) -> dict`, 对每个工况调用 `solve_one_regime`
- ☐ SLSQP 不收敛或约束不满足时, 切换 PSO/GA 全局备选求解 (`scipy.optimize.differential_evolution` 或 `pyswarms`), 记录算法名称、收敛标志、迭代次数
- ☐ 不可行工况标记 `success=False` 并提示需放宽边界或硬件改造; 输出各工况最优参数表, 所有工况满足 $C_{out} \leq C_{limit}$ (对应 spec 4.2 规则 5/6、design 2.1.3 分支)

5. 问题3 灵敏度分析 (sensitivity 模块)

5.1 实现数值雅可比 numeric_jacobian

- ☐ 在 `sensitivity/jacobian.py` 实现 `numeric_jacobian(model, x0, regime, step=0.01) -> dict`, 有限差分计算 $S_{U_i}^C = \partial C_{out} / \partial U_i, S_{T_i}^C = \partial C_{out} / \partial T_i, S_{U_i}^P = \partial P_{total} / \partial U_i, S_{T_i}^P = \partial P_{total} / \partial T_i$
- ☐ 步长取变量范围 1%, 做步长减半验证一致性, 不一致时打印告警 (对应 spec 5.3 风险应对)
- ☐ 返回 4x4 灵敏度矩阵 + 步长减半一致性指标

5.2 实现优先级判定 `priority_rule`

- ☐ 在 `sensitivity/priority.py` 实现 `priority_rule(sens) -> dict`，计算性价比比值 $|S^C/S^P|$ （单位电耗带来的浓度下降），比值大者优先调整
- ☐ 输出"优先调电压"与"优先调振打"的判定规则文本，明确何种工况优先调电压、何种优先调振打（对应 spec 4.3 规则 4）

5.3 实现两差异工况对比 `compare_two_regimes`

- ☐ 在 `sensitivity/compare.py` 实现 `compare_two_regimes(rA, rB, model) -> dict`，从问题 2 工况中选取 2 个差异明显工况（如最高浓度 vs 最低浓度）
- ☐ 输出 2 张完整操作参数表 ($U_{1\sim4}$ 、 $T_{1\sim4}$ 、 P_{total} 、 C_{out} 预测)
- ☐ 从物理机理（高浓度需更高电压/更短振打周期）+ 数据特征两层面分析差异原因，输出含机理解释与数据支撑的差异原因文本（对应 spec 4.3 规则 3）

6. 问题4 收紧分析 (tighten 模块)

6.1 实现收紧重求解 `resolve_under_limit`

- ☐ 在 `tighten/resolve.py` 实现 `resolve_under_limit(regimes, model, C_limit_new=5.0) -> dict`，复用 `optim.solve_all_regimes` 传入 $C'_{limit} = 5$ 重求解各工况
- ☐ 至少含高浓度工况；返回各工况收紧后最优参数与最低电耗 $P^*(5)$ （对应 spec 4.4 规则 1、design 2.1.3 扩展点）

6.2 实现电耗增幅计算 `delta_power`

- ☐ 在 `tighten/resolve.py` 实现 `delta_power(P10, P5) -> dict`，按 $\Delta P\% = (P^*(5) - P^*(10))/P^*(10) \times 100\%$ 计算各工况及整体总电耗增加百分比
- ☐ 输出各工况及整体电耗增加百分比表（对应 spec 4.4 规则 2）

6.3 实现可行性校验 `feasibility_check`

- ☐ 在 `tighten/resolve.py` 实现 `feasibility_check(sol, bounds) -> dict`，校验收紧后最优参数在物理可行域（电压 $\leq$ 变压器上限、振打 $\geq$ 机械下限）
- ☐ 输出各工况参数可行性与不可行工况清单，不可行工况明确提示需硬件改造或放宽电压上限（对应 spec 4.4 规则 4、spec 5.4 风险应对）

6.4 实现高浓度工况应对建议 `high_conc_advice`

- ☐ 在 `tighten/advice.py` 实现 `high_conc_advice(regime, sol10, sol15) -> str`，基于收紧前后最优参数差异给出具体可操作建议
- ☐ 建议含：电压提升方向、振打周期调整方向、多电场协同策略（如前电场提电压承担主要除尘负荷、后电场精细控制）
- ☐ 建议必须基于寻优结果与物理机理，不泛泛而谈（对应 spec 4.4 规则 3、禁止项）

7. 输出与可视化 (report 模块)

7.1 实现关系曲线图 `plot_relation_curves`

- ☐ 在 `report/plots.py` 实现 `plot_relation_curves(model, df, out_dir) -> None`，绘制入口温度/流量/浓度与 C_{out} 关系曲线、 U_i 与效率 η_i 关系曲线、振打周期与峰值 C_{peak} 关系曲线
- ☐ 输出 PNG 到 `Problem/outputs/`，图标题、轴标签使用中文，字体颜色统一非纯黑

7.2 实现工况散点图 `plot_regime_scatter`

- ☐ 在 `report/plots.py` 实现 `plot_regime_scatter(regimes, df, out_dir) -> None`，在 $C_{in} - T_{in}$ 二维平面绘制 K-Means 工况划分散点，按工况着色并标注边界与中心

7.3 实现参数对比图 `plot_param_compare`

- ☐ 在 `report/plots.py` 实现 `plot_param_compare(rA, rB, out_dir) -> None`，绘制两差异工况 $U_{1\sim4}$ 、 $T_{1\sim4}$ 、 P_{total} 对比柱状图

7.4 实现灵敏度热力图 `plot_sensitivity_heatmap`

- ☐ 在 `report/plots.py` 实现 `plot_sensitivity_heatmap(sens, out_dir) -> None`，绘制 S^C 、 S^P 及性价比比值 $|S^C/S^P|$ 热力图

7.5 实现电耗增幅图 `plot_delta_power`

- ☐ 在 `report/plots.py` 实现 `plot_delta_power(dp, out_dir) -> None`，绘制各工况收紧前后 P_{total} 对比柱状图与 $\Delta P\%$ 增幅柱状图

7.6 实现 Markdown 表格生成 `to_markdown_tables`

- ☐ 在 `report/tables.py` 实现 `to_markdown_tables(results) -> str`，汇总四个问题结果为论文用 Markdown 表格字符串
- ☐ 表格含：问题1 关系分析与特征重要性表、问题2 各工况最优参数表、问题3 两工况对比与优先级规则表、问题4 电耗增幅与可行性表
- ☐ 所有数学公式与变量用 `$$...$$` 包裹；输出到 `Problem/outputs/results.md` 供论文直接粘贴

8. 主流程编排与端到端串联

8.1 实现主入口 `main.py`

- ☐ 在 `Problem/main.py` 实现 `run_pipeline()`，按 `design.md` 2.1.3 活动图顺序串联：加载→清洗→拟合电耗→拟合 Deutsch→特征重要性→工况划分→问题2 寻优→问题3 灵敏度→问题4 收紧→导出报告
- ☐ 各步骤打印日志进度与关键指标 (R^2 、各工况 C_{out} 、 $\Delta P\%$)
- ☐ 支持 `python main.py --config config/config.yaml` 命令行启动；可选 `--skip-mysql` 跳过数据库持久化
- ☐ 全流程固定随机种子 `seed=42`，保证可复现

9. 测试与验证

9.1 单元测试

- ☐ 在 `tests/test_data_loader.py` 测试 `load_raw` 列数/行数/dtype 校验、`clean_and_impute` 缺失值插补后无缺失、边界统计正确性
- ☐ 在 `tests/test_modeling.py` 测试 `fit_power_model` 返回 $k_i \geq 0$ 且 R^2 合理、`predict_cout` 单调性 (U_i 增则 C_{out} 减、 T_i 增则 C_{out} 增)
- ☐ 在 `tests/test_optim.py` 测试 `solve_one_regime` 满足 $C_{out} \leq C_{limit}$ 与各电场边界约束、不可行工况返回 `success=False`
- ☐ 在 `tests/test_sensitivity.py` 测试 `numeric_jacobian` 步长减半一致性、`priority_rule` 输出格式正确
- ☐ 在 `tests/test_tighten.py` 测试 `delta_power` 公式正确性、`feasibility_check` 不可行工况识别

9.2 集成测试

- ☐ 在 `tests/test_pipeline.py` 用前 1440 行 (1 天) 子集跑完整 `run_pipeline`，验证各模块衔接无报错、产物文件齐全
- ☐ 验证问题 4 复用问题 2 框架 (`solve_one_regime` 传不同 C_{limit}) 结果可比

9.3 端到端验证

- ☐ 用全量 10080 行数据跑 `python main.py`，确认 `outputs/` 下生成所有 PNG 图表与 `results.md`
- ☐ 核对问题 2 所有工况 $C_{out} \leq 10 \text{ mg/Nm}^3$ 、问题 4 所有可行工况 $C_{out} \leq 5 \text{ mg/Nm}^3$
- ☐ 核对电耗模型 R^2 、Deutsch 拟合残差、灵敏度步长一致性指标均达标，不达标在报告中明确标注不确定性

10. 部署与配置

10.1 环境配置

- ☐ 编写 `Problem/README.md` 运行说明：Python 版本 (≥ 3.9)、`pip install -r requirements.txt`、MySQL 可选配置
- ☐ 在 `config/config.yaml` 集中所有可调参数，运行时只读配置不改代码

10.2 数据库初始化

- ☐ 提供 `scripts/init_db.py` 执行 `config/db_schema.sql` 创建 MySQL 表结构，便于复算与论文数据回溯
- ☐ MySQL 不可达时自动降级本地 parquet 缓存，打印告警不阻断

10.3 复现性保障

- ☐ 全流程固定 `seed=42`，`requirements.txt` 固定依赖版本
- ☐ 在 `outputs/` 同时导出 `run_meta.json` 记录运行时间、依赖版本、随机种子、配置快照，供论文附录引用

11. 审查与验证

11.1 代码审查

- ☐ 审查各模块接口签名与 `design.md` 2.2.2 接口清单一致，参数 dict 字段向后兼容
- ☐ 审查物理约束落实： $k_i \geq 0$ 、效率单调性、各电场独立边界、振打双重约束、固定种子

11.2 设计回顾

- ☐ 核对 `spec.md` 4.1–4.4 所有业务规则与验收条件均有对应任务覆盖
- ☐ 核对 `spec.md` 5.5 关键反馈意见（机理为主/数据为辅、各电场独立、外推不确定性标注、振打峰值估算标注）在代码与报告中落实

11.3 变更确认

- ☐ 确认最终产物清单：`outputs/` 下 PNG 图表（关系曲线、工况散点、参数对比、灵敏度热力、电耗增幅）、`outputs/results.md`（论文用 Markdown 表格）、`outputs/run_meta.json`（复现元数据）
- ☐ 确认所有 $\$$ 数学公式在 `results.md` 中正确渲染，所有图表字体颜色统一非纯黑、缩进符号一致